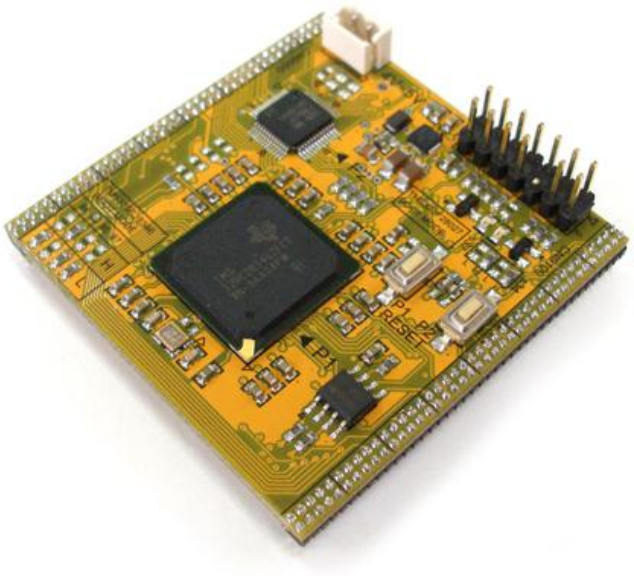
TMS320C28346 듀오27 모듈

TMS320F28027 Based

Programmable ADC Manual



SyncWorks

TMS320C2000 Specialist

목차

1 TMS320C28346 듀오 27 모듈 ADC 모드 4

1.1 TMS320F28027 Based Programmable ADC 주요성능 4

1.2 TMS320C28346 과 TMS320F28027 간 통신 연결 5

1.3 TMS320F28027 Based Programmable ADC 운용 개념 6

1.4 TMS320F28027 Based Programmable ADC 용 API 7

1.4.1 init_Mcbspa_Gpio 함수 7

1.4.2 init_mcbsp_spi_slave 함수 8

1.4.3 ADC_Cnf 함수 9

1.4.1 Filter_Cnf 함수 11

1.4.2 ADC_Run 함수 13

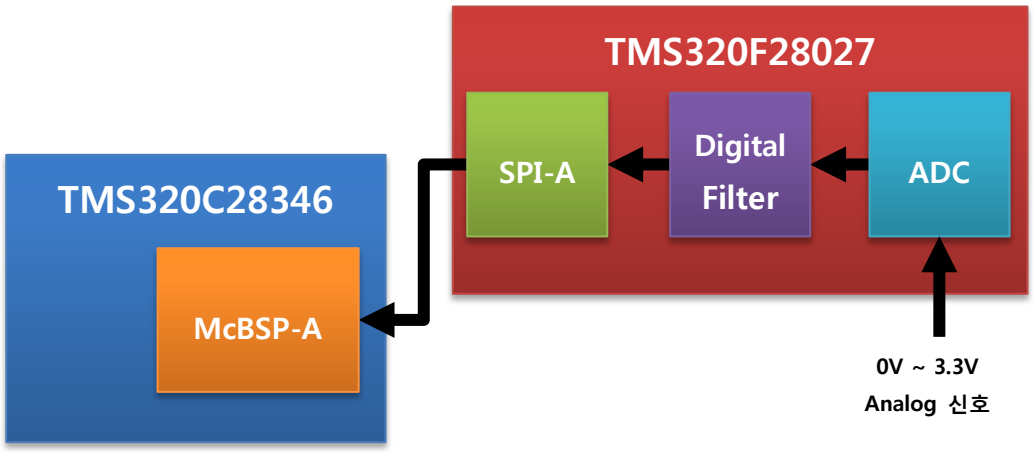
2 ADC 예제 따라하기 (CCSv3.3 기준) 14

Revision History

날 짜	내 용
11. 10. 17.	- Rev. 1.1, 매뉴얼 신규 작성

1 TMS320C28346 듀오27 모듈 ADC 모드

TMS320C28346 – 300MHz/600MFLOPS 고성능 MCU에는 안타깝게도 ADC가 내장되어 있지 않습니다. 그렇다고 외장 ADC를 달자니, TMS320C2000 특유의 스마트한 ADC를 사용하지 못해서 안타깝고... 이에 TMS320F28027 MCU를 추가 탑재 시켜 ADC 문제를 아주 스마트하게 해결했습니다. 샘플링 주파수, 변환 채널 등의 초기화 데이터만 전송하면 TMS320F28027이 끊임없이 ADC 결과를 TMS320C28346으로 전송합니다. ADC만 하기에는 TMS320F28027의 연산자원이 남아돌기에 디지털 필터 18종을 탑재시켜 노이즈를 쉽게 걸러 낼 수 있도록 설계했습니다.



[그림 1] TMS320F28027 ADC & Digital filter 모드 운용 블록도

1.1 TMS320F28027 Based Programmable ADC 주요성능

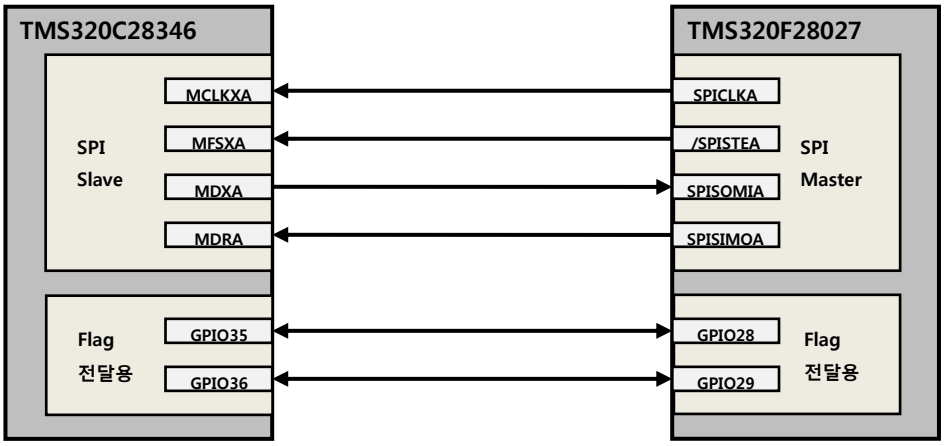
TMS320C28346 듀오27 모듈은 TMS320F28027를 ADC로 사용하기 때문에 ADC의 성능은 TMS320F28027의 내부 ADC 성능과 동일합니다. 하지만 ADC 결과를 SPI 통신을 통해서 TMS320C28346에 전송해야 하기 때문에 ADC 최대 샘플링 속도가 TMS320F28027의 SPI 최대 통신속도(15Mbps)를 넘지 못합니다. 이를 고려하면 TMS320F28027를 통한 ADC 주요 성능은 다음과 같이 정리 됩니다. 자세한 전기스펙은 TMS320F28027의 데이터 매뉴얼을 참고하기 바랍니다.

ADC 입력 채널 수	13채널
ADC 분해능	12비트
ADC 최대 샘플링 속도	약 1MHz
ADC 최저 샘플링 속도	약 0.1Hz
ADC 신호 입력 범위	0V ~ 3.3V

[그림 1-1] TMS320F28027 ADC 모드 주요 성능

1.2 TMS320C28346과 TMS320F28027간 통신 연결

TMS320C28346 듀오27 모듈에 탑재된 TMS320F28027을 ADC로 운용하기 위한 설정 데이터와 결과 데이터를 송수신 하기 위해서 TMS320C28346과 TMS320F28027은 그림 1-2와 같이 통신 인터페이스로 연결되어 있습니다.



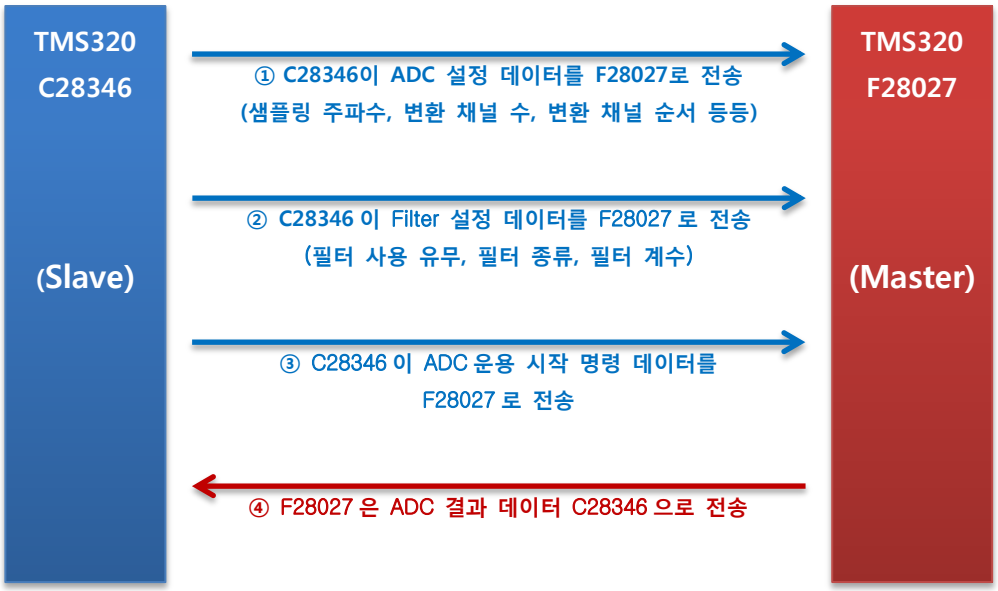
[그림 1-2] TMS320C28346과 TMS320F28027 통신 연결도

기본적으로 TMS320C28346과 TMS320F28027은 SPI 통신으로 데이터를 주고 받습니다. 이때 TMS320C28346에서는 McBSP A포트를 SPI 모드로 운용하고 TMS320F28027 에서는 SPI A포트를 사용하고 있습니다. 이때 TMS320C28346은 SPI Slave이고, TMS320F28027이 SPI Master 인 것에 주의 해야 합니다.

그 외 TMS320C28346의 GPIO35, 36이나 TMS320F28027의 GPIO28, 29은 플래그 신호 전달용 혹은 여러가지 범용적 용도로 사용할 수 있도록 연결되어 있습니다.

1.3 TMS320F28027 Based Programmable ADC 운용 개념

TMS320C28346 듀오27 모듈의 TMS320F28027 기반 Programmable ADC는 그림 1-3 과 같은 개념으로 운용됩니다. 통신상의 마스터가 되는 TMS320F28027 쪽 펌웨어는 싱크웍스에서 직접 제품에 탑재해 드립니다. 사용자는 TMS320F28027을 ADC로 운용하기 위해서 '①, ②, ③' 과정을 TMS320C28346에서 수행해야 합니다. 이를 위한 TMS320C28346 예제도 싱크웍스에서 제공하니 사용자는 아주 쉽게 TMS320F28027을 ADC로 활용하실 수 있습니다.



[그림 1-3] TMS320C28346 듀오27 모듈 ADC 모드 운용 개념도

1.4 TMS320F28027 Based Programmable ADC용 API

싱크웍스는 TMS320C28346 듀오27 모듈에서 외부 ADC로 동작하는 TMS320F28027을 제어하기 위해 ADC28027 API를 제공합니다. 사용자는 ADC28027 API를 사용해서 다음과 같은 몇 가지 함수의 사용만으로 손쉽게 TMS320F28027을 ADC로 운용할 수 있습니다. ADC28027 API와 예제는 싱크웍스 홈페이지 기본예제 게시판에서 다운로드 받을 수 있습니다. **>> 싱크웍스 기본예제 게시판 바로가기**

ADC28027 API 파일명>> DSP28x_ADC28027.c, DSP28x_ADC28027.h

함수명	입력값
init_Mcbspa_Gpio()	없음
init_mcbsp_spi_slave()	없음
ADC_Cnf()	SamFreq, MaxConv, ChSel[], AcqPs
Filter_Cnf()	FilterType, FilterCoeffSel
ADC_Run()	FilterEnable

[그림 1-4] ADC28027 API가 제공하는 함수들

각 함수들에 대한 상세한 설명은 다음과 같습니다.

1.4.1 init_Mcbspa_Gpio 함수

Description:

TMS320C28346 듀오27 모듈에서 TMS320C28346과 TMS320F28027간의 통신을 위해서 TMS320C28346의 GPIO20, 21, 22, 23 핀을 McBSP 기능으로 설정하는 함수 입니다.

Function Prototype:

```
/*-----------------------------------------------------------------------------
  DSP28x_ADC28027.c
-----------------------------------------------------------------------------*/

void Init_Mcbspa_Gpio(void);
```

Input Values:

없음

Return Values:

없음

1.4.2 init_mcbsp_spi_slave 함수

Description:

TMS320C28346 듀오27 모듈에서 TMS320C28346과 TMS320F28027간의 통신을 위해서 TMS320C28346의 McBSP를 SPI 모드로 운용하기 위한 초기화 함수 입니다.

Function Prototype:

```
/*-------------------------------------------------------------------------
  DSP28x_ADC28027.c
-------------------------------------------------------------------------*/

void init_mcbsp_spi_slave(void);
```

Input Values:

없음

Return Values:

없음

1.4.3 ADC_Cnf 함수

Description:

TMS320C28346 듀오27 모듈에서 TMS320F28027이 ADC로 운용되기 위한 주요 설정 값 (샘플링 주파수, 변환 채널 수, 변환 채널 순서, S/H 윈도우 타임)을 전송하는 함수 입니다.

Function Prototype:

```
/*-----------------------------------------------------------------------------
  DSP28x_ADC28027.c
-----------------------------------------------------------------------------*/

void ADC_Cnf(float32 SamFreq, Uint16 MaxConv, Uint16 *ChSel, Uint16 AcqPs);
```

Input Values:

SamFreq (32bit Float Type):

- 샘플링 주파수 설정, Hz 단위로 0.1 ~ 1,000,000까지 설정 가능함.

MaxConv (16bit Unsigned Integer Type):

- 변환 채널 수 설정, 1~16까지 설정 가능함.

***ChSel** (16bit Unsigned Integer Pointer Type):

- 변환 채널 순서 설정, 16 포인트 배열의 시작주소를 포인터로 입력함.

AcqPs (16bit Unsigned Integer Type):

- ADC Sample/Hold 타이밍 설정. ADC Clock Cycle 단위로 7~64까지설정가능함. (이때 1 ADC Clock Cycle = 약 16.67nsec)

Return Values:

없음

Example:

```
. . .
// ADC 샘플링 주파수(Hz 단위): 20KHz
Float32 Sam_Freq = 20000;

// ADC 변환 채널 수: 4ch 변환
Uint16 MaxConv_Num = 4;

// ADC 변환 채널 순서 (2ch → 1ch → 3ch → 4ch)
Uint16 Ch_Order[16] = {2,1,3,4,0,0,0,0,0,0,0,0,0,0,0,0};

// ADC Acq window 설정 : 7 ADC Clock Cycle(7/60MHz)
Uint16 Acq_Ps = 7;
. . .
// ADC 초기화 함수 실행
ADC_Cnf(Sam_Freq, MaxConv_Num, Ch_Order, Acq_Ps);
. . .
```

1.4.1 Filter_Cnf 함수

Description:

TMS320C28346 듀오27 모듈에서 TMS320F28027이 ADC 운용 시 디지털 필터를 함께 운용하기 위한 설정값(필터 타입, 필터 Cutoff 주파수)을 전송하는 함수 입니다.

Function Prototype:

```
/*-------------------------------------------------------------------------
  DSP28x_ADC28027.c
-------------------------------------------------------------------------*/

void Filter_Cnf(Uint16 FilterType, Uint16 FilterCoeffSel);
```

Input Values:

FilterType (16bit Unsigned Integer Type):

- 필터 타입 설정, 두 가지 타입의 필터 중 한가지를 선택할 수 있음.
  - 0 값으로 설정 시: Low Pass Filter 타입으로 설정됨 .
  - 1 값으로 설정 시: High Pass Filter 타입으로 설정됨.

FilterCoeffSel (16bit Unsigned Integer Type):

- 필터의 Cutoff 주파수 설정, 0~8까지 설정할 수 있음. 이때 **Cutoff 주파수는 "[(설정값 + 1)/10] * ADC 신호 대역폭"** 임.
- 예를 들어, ADC 샘플링 주파수가 20KHz라면, ADC 신호 대역폭은 1/2인 10KHz 임. 이때 'FilterCoeffSel = 4' 라면, 필터의 Cufoff 주파수는 5KHz 임.

Return Values:

없음

Example:

```
. . .
// Filter 타입 선택 (0: LPF, 1: HSP)
Uint16 Filter_Type = 0;

// Filter Cufoff 주파수 설정 ( 0 ~ 8 : ((설정값 + 1)/10)*ADC신호대역폭 )
Uint16 Filter_CoeffSel = 0;
. . .
// Filter 초기화 함수 실행
Filter_Cnf(Filter_Type, Filter_CoeffSel);
. . .
```

1.4.2 ADC_Run 함수

Description:

TMS320C28346 듀오27 모듈에서 TMS320F28027이 ADC로 운용되기 위한 설정과 필터 설정이 완료된 후에 ADC 운용 시작을 지시하는 함수 입니다.

Function Prototype:

```
/*-----------------------------------------------------------------------------
  DSP28x_ADC28027.c
-----------------------------------------------------------------------------*/

void ADC_Run(Uint16 FilterEnable);
```

Input Values:

FilterEnable (16bit Unsigned Integer Type):

- 필터 사용 여부 설정
  - 0 값으로 설정 시: 필터를 사용하지 않음.
  - 1 값으로 설정 시: 필터를 사용함.

Return Values:

없음

Example:

```
. . .
// Filter 사용 여부 선택 (0: Disable, 1: Enable)
Uint16 Filter_Enable= 0;

. . .
// ADC 운용 시작 함수 실행
ADC_Run(Filter_Enable);
. . .
```

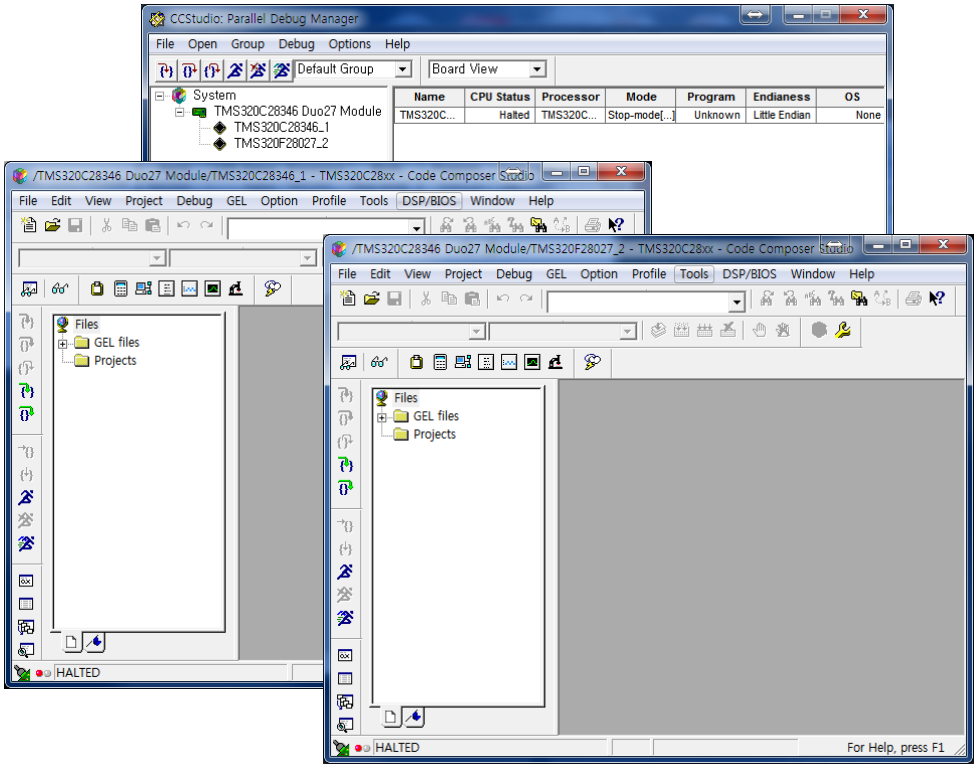
2 ADC 예제 따라하기 (CCSv3.3 기준)

싱크웍스 홈페이지에서 TMS320C28346 듀오27 모듈의 외부 ADC 관련 API와 예제를 다운로드 받았다면, 아래의 따라하기 과정을 통해서 직접 TMS320F28027를 외부 ADC 로 운용 해보시기 바랍니다. 여기서는 TDS510USB-C2K 에뮬레이터와 CCSv3.3 버전을 기준으로 설명합니다.

Step1. TMS320C28346 듀오27 모듈 접속하기

TMS320C28346 듀오27 모듈의 TMS320C28346과 TMS320F28027에 에뮬레이터와 CCS를 통해서 동시에 접속합니다. TMS320C28346 듀오27 모듈을 위한 CCS 설정법은 다른 링크의 내용을 참고 하기기 바랍니다.

TMS320C28346 듀오27 모듈 CCS 설정법 >> 바로가기

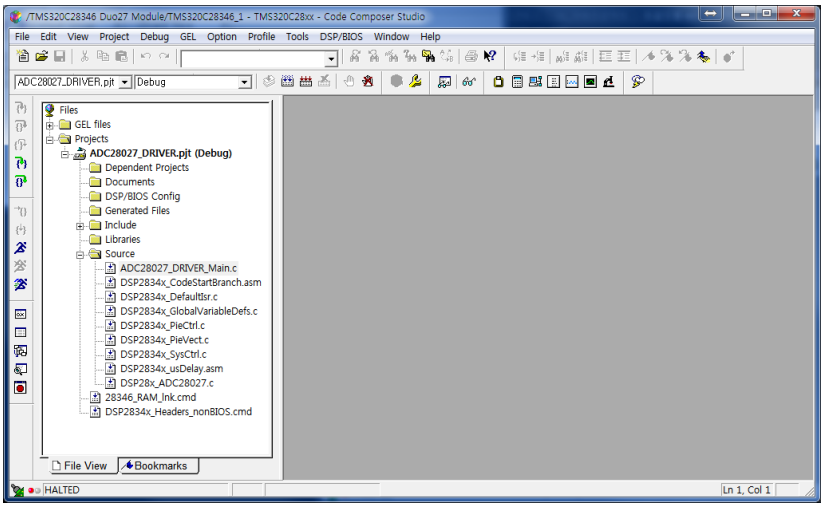


[그림 2-1] CCS에서 TMS320C28346 듀오27 모듈의 각 프로세서에 접속한 모습

Step2. TMS320C28346에 접속한 CCS에서 예제열기

두 개의 CCS 창 중에서 **TMS320C28346**에 접속한 CCS에서 ADC28027_DRIVER 프로젝트를 불러옵니다. CCS의 상단메뉴 Project → Open을 선택하고, 다음 경로에서 프로젝트 파일을 찾아서 불러들입니다.

파일경로> `C:\SyncWorks\TMS320C28346_DUO27\ADC28027_DRIVER_V330\ADC28027_DRIVER.pjt`

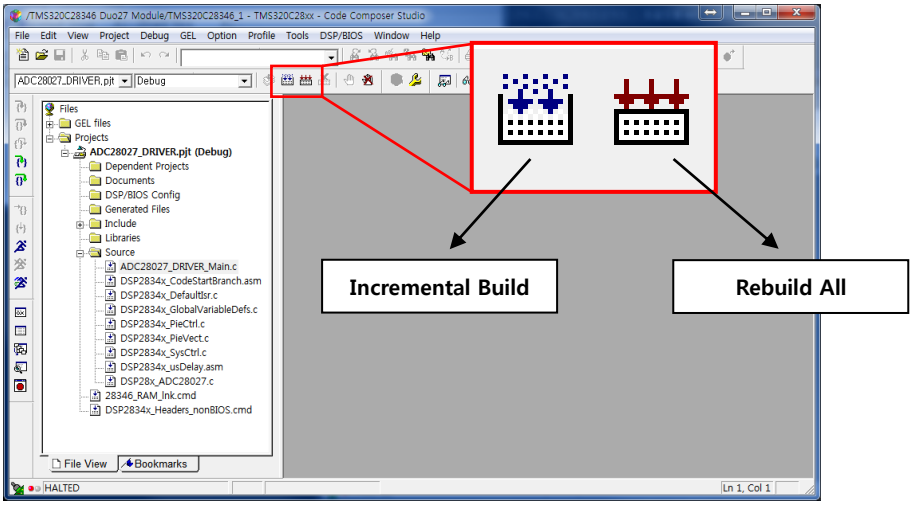


[그림 2-2] ADC28027_DRIVER 프로젝트 열기

CCS화면에서 그림 2-2와 같이 File view 박스에 ADC28027_DRIVER.pjt가 불러들여졌음을 확인합니다.

Step3. ADC28027_DRIVER 프로젝트를 Build 합니다.

CCS 상단메뉴 Project → build을 선택하거나, Build 아이콘을 클릭하여 프로젝트를 빌드 합니다.



[그림 2-3] build 아이콘

Step4. ADC28027_DRIVER.out 파일을 로드 합니다

CCS의 Customize 설정에서 **Load Program After Build** 옵션을 체크했다면, Build 과정이 완료되면 out 파일이 자동으로 프로세서에 로드 됩니다.

Load Program After Build 옵션이 체크돼있지 않았다면, CCS 상단메뉴 File → Load Program을 선택하고 다음 경로의 결과 파일을 직접 로드 해야 합니다.

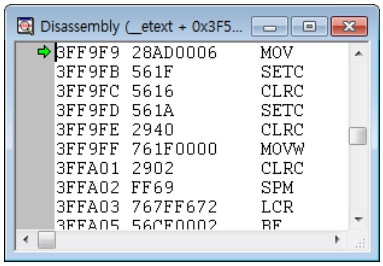
파일경로>

```
C:\SyncWorks\TMS320C28346_DUO27\ADC28027_DRIVER_V330\Debug\ADC28027_DRIVER.out
```

Step5. TMS320C28346 프로세서를 Reset 합니다.

CCS 상단 메뉴 Debug → Reset CPU를 선택합니다.

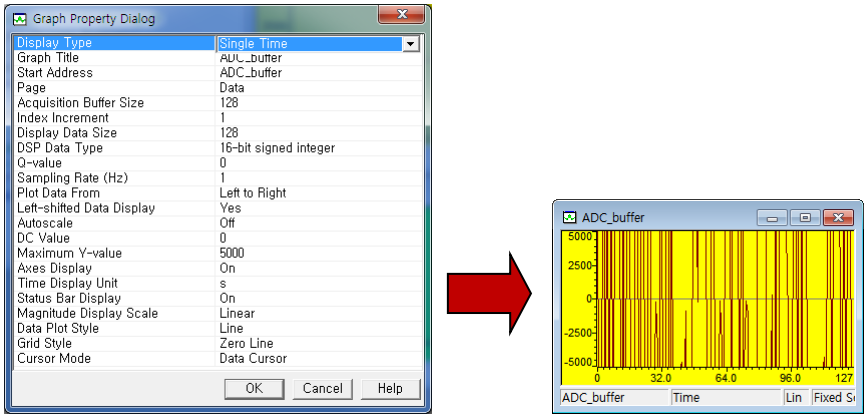
그림 2-4와 같이 Disassembly 창의 녹색 화살표가 0x3FF9F9 번지를 가리키는 것으로 프로세서 리셋을 확인합니다.



[그림 2-4] 프로세서 Reset 확인

Step6. ADC_buffer 데이터를 그래프로 관찰합니다.

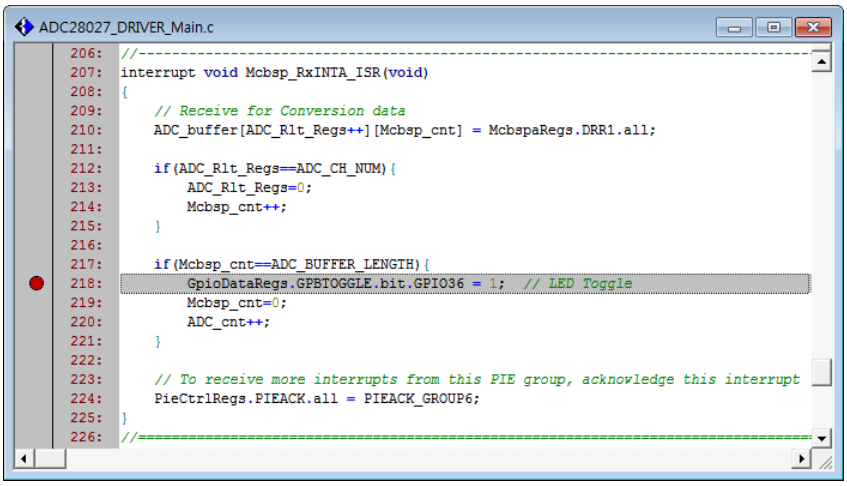
CCS 상단 메뉴 View → Graph → Time/Frequency 를 선택합니다. Graph Property Dialog창이 나타나면 그림 2-5와 같이 설정합니다. 설정이 완료되면 OK 버튼을 클릭해서 ADC_buffer 데이터를 그래프로 관찰해 봅니다. ADC_buffer 에는 아직 의미 있는 데이터가 없기 때문에 그림 2-5와 같이 쓰레기 데이터만 보이는 상태입니다.



[그림2-5] ADC_buffer 데이터를 그래프로 관찰

Step7. Mcbsp_RxINT_ISR() 함수안에 브레이크포인트를 설정합니다.

ADC28027_DRIVER_Main.c 파일을 열어서 Mcbsp_RxINT_ISR() 함수 안에 그림 2-6과 같은 위치에 브레이크 포인트를 설정합니다.

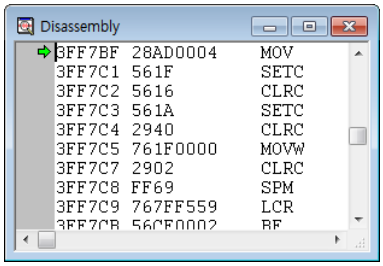


[그림 2-6] Mcbsp_RxINT_ISR() 함수안에 브레이크 포인트 설정

보충:
브레이크 포인트를 설정하려면, 코드 에디터 창에서 원하는 위치에 커서를 위치시키고 마우스 오른쪽 클릭 후 Toggle Software Breakpoint를 선택하시면 됩니다.
혹은 CCS 상단 아이콘 중  와 같은 손바닥 모양의 아이콘을 사용해도 설정할 수 있습니다.

Step8. TMS320F28027 프로세서를 Reset 합니다.

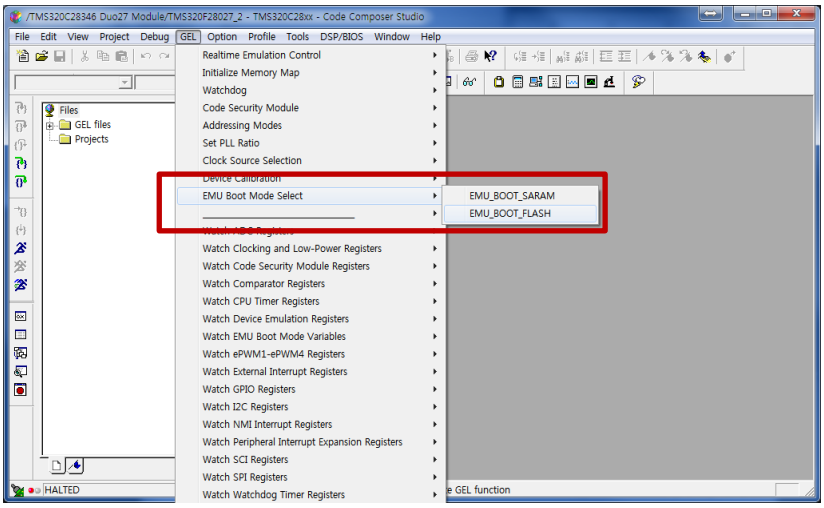
두 개의 CCS 창 중에서 **TMS320F28027**에 접속한 CCS를 열고, CCS 상단 메뉴 Debug → Reset CPU를 선택합니다. 그림 2-7와 같이 Disassembly 창의 녹색 화살표가 0x3FF7BF 번지를 가리키는 것으로 프로세서 리셋을 확인합니다.



[그림 2-7] 프로세서 Reset 확인

Step9. TMS320F28027의 부트모드를 설정합니다.

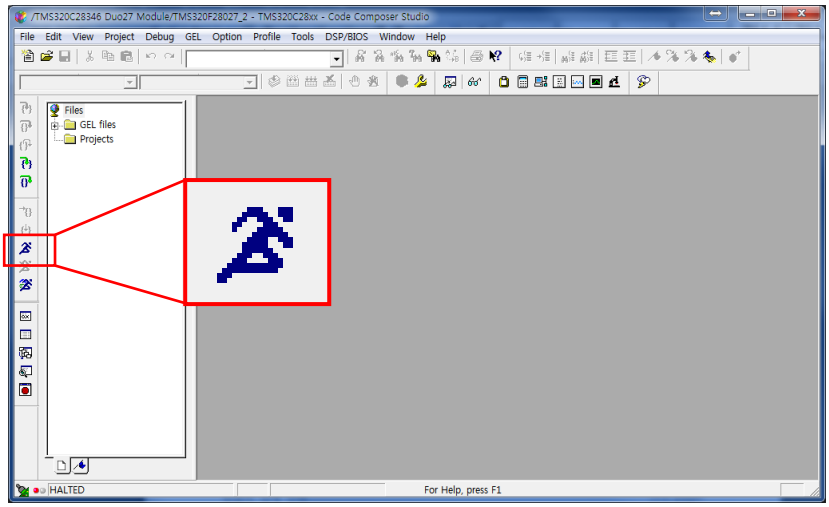
TMS320F28027에 접속한 CCS에서 CCS 상단 메뉴 Gel → Emu Boot Mode Select → EMU_BOOT_FLASH 를 선택합니다.



[그림 2-8] TMS320F28027의 플래시 부트 모드 설정

Step10. TMS320F28027를 실행 시킵니다.

TMS320F28027에 접속한 CCS 상단메뉴 Debug → Run을 선택하거나 그림 5-9에 표시된 아이콘을 클릭하여 TMS320F28027을 실행 시킵니다.



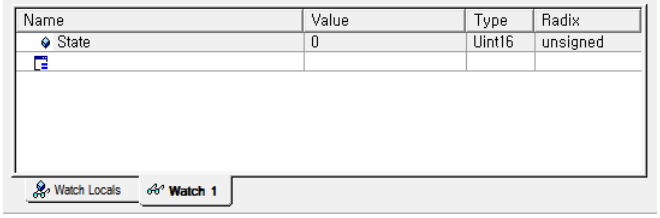
[그림 2-9] Run 아이콘

Step11. TMS320C28346를 실행 시킵니다.

TMS320C28346에 접속한 CCS 상단메뉴 Debug → Run을 선택하거나 Run 아이콘을 클릭하여 프로세서에 로드 된 예제 프로그램을 실행합니다.

Step12. State 변수를 Watch Window에 등록합니다.

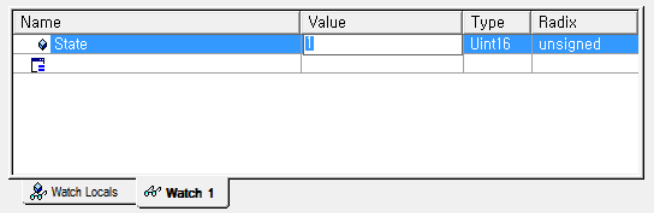
TMS320C28346에 접속한 CCS 상단 메뉴 View → Watch Window 를 선택합니다. Watch Window가 나타나면 Watch1 탭에 State 변수를 등록합니다.



[그림 2-10] Watch Window에 State 변수 등록

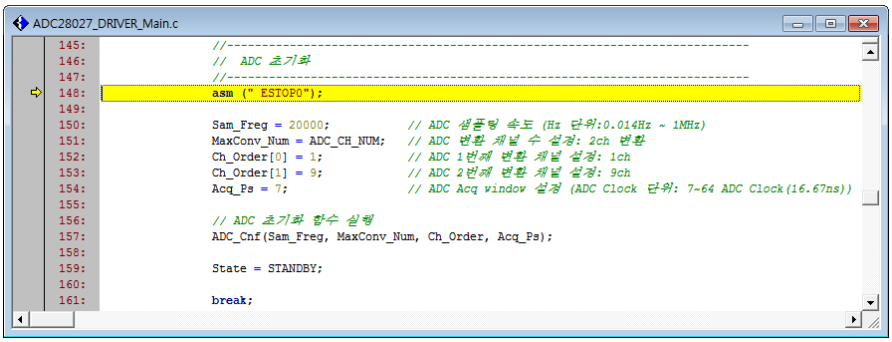
Step13. State 변수를 Watch Window에 변경합니다.

Watch Window에서 방금 등록한 State 변수 값을 1로 수정합니다.



[그림 2-11] Watch Window에 State 변수 값 변경

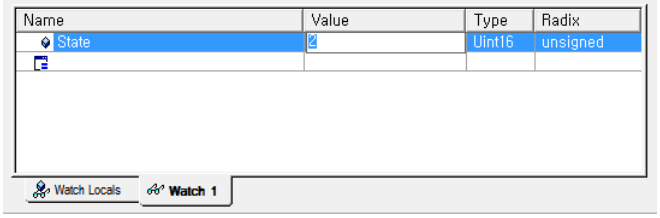
State 값을 1로 수정한 다음 그림 2-12와 같이 프로그램 실행이 멈춘 것을 확인합니다. ADC가 어떻게 설정 됐는지 확인하고, 프로그램을 다시 Run 합니다.



[그림 2-12] ADC 초기화

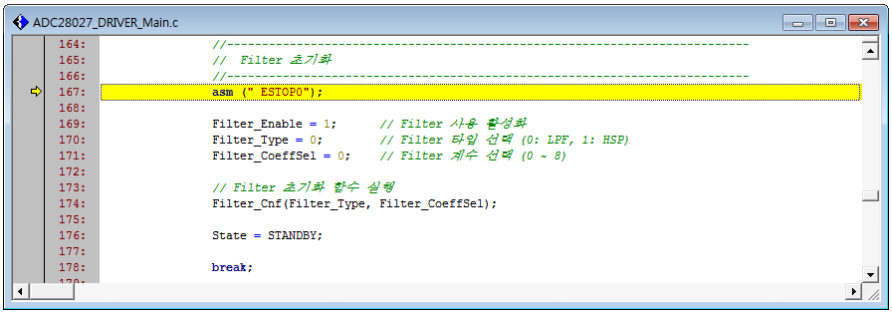
Step14(옵션). State 변수를 Watch Window에서 변경합니다.

Digital Filter를 사용하지 않고 순수 ADC 만 할 것 이라면 이 단계(Step14)는 건너 뜁니다. 만일 Digital Filter를 사용할 것이라면 State 변수 값을 2로 수정합니다.



[그림 2-13] Watch Window에 State 변수값 변경

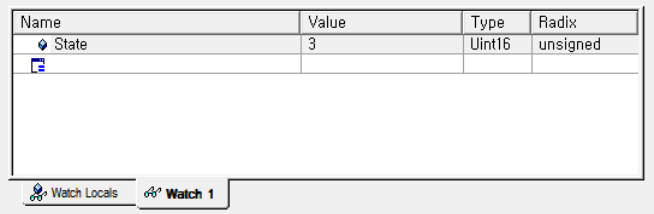
State 값을 2로 수정한 다음 그림 2-14와 같이 프로그램 실행이 멈춘 것을 확인합니다. Filter가 어떻게 설정 됐는지 확인하고, 프로그램을 다시 Run 합니다.



[그림 2-14] Filter 초기화

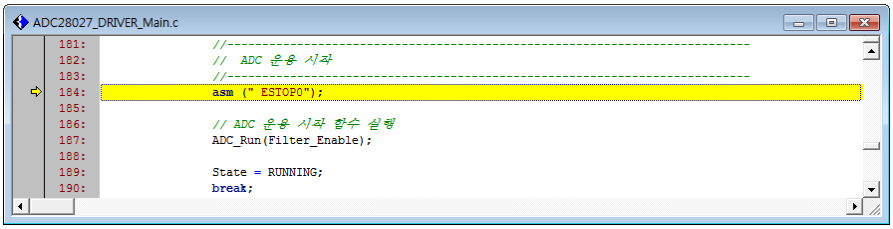
Step15. State 변수를 Watch Window에서 변경합니다.

Watch Window에서 State 변수 값을 3로 수정합니다.



[그림 2-15] Watch Window에 State 변수 값 변경

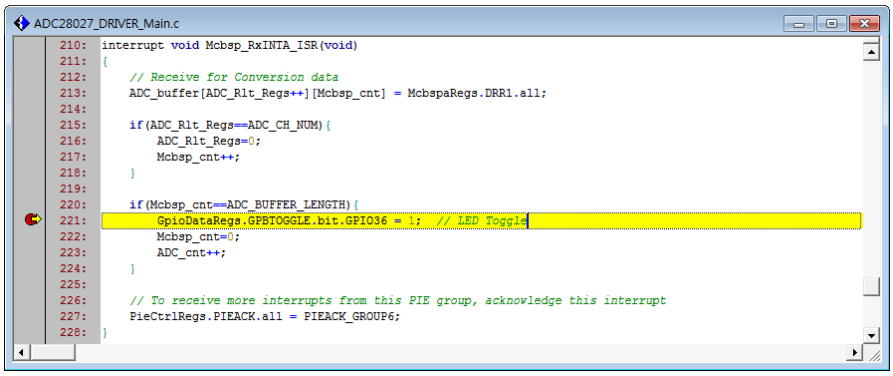
State 값을 3으로 수정한 다음 그림 2-16와 같이 프로그램 실행이 멈춘 것을 확인합니다. 예제에서의 설정이 어떻게 되어 있는지 확인하고, 프로그램을 다시 Run 합니다.



[그림 2-16] ADC 운용 시작

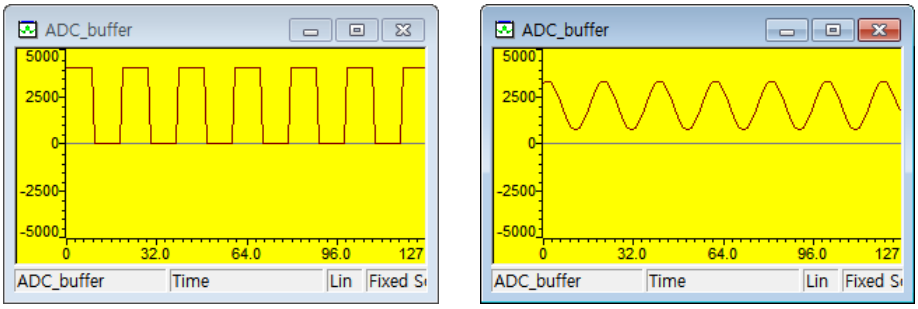
Step16. ADC 결과를 그래프로 확인합니다.

프로그램이 Mcbsp_RxINT_ISR() 함수 안에 앞서 설정한 브레이크 포인트에서 멈춘 것을 확인합니다.



[그림 2-17] 프로그램 정지 확인

앞서 띄웠던 ADC_buffer 그래프를 확인합니다. 그림 2-17은 ADC 입력 1번 채널에 1KHz, 50% Duty의 PWM 신호를 입력 했을 때의 ADC 결과 입니다. 사용자가 ADC 입력 채널에 입력하는 신호에 따라서 ADC_buffer 그래프에 보이는 결과는 달라질 수 있습니다.



[그림 2-18] ADC 결과 확인 – 1KHz, 50% Duty PWM 신호의 예

(좌측: 필터 사용안함, 우측: 필터 사용함)